

Assignment 2

Probabilistic Simulation

Deadline: Friday October 16th, 23:59

1 Non-Uniform Eulerian Systems: Cylinder Flow Dataset [50pt]

1.1 Task Description

For the first task, you will model the future evolution of a two-dimensional fluid flow around a cylinder. You can access the notebook for this task [here](#).

The flow field is represented on an **unstructured, spatially nonuniform triangular mesh**. Your task is to construct and compare two different approaches for predicting the future velocity field:

1. a **deterministic model**, trained to predict a single future state; and
2. a **conditional flow-matching model**, trained to represent a distribution of possible future states.

The main objective is to investigate the difference between deterministic point prediction and distributional prediction for dynamical systems.

The neural-network architecture is not prescribed. You may choose an appropriate architecture and representation for the supplied unstructured mesh.

1.2 Dataset

The dataset consists of multiple independent trajectories of two-dimensional flow around a cylinder.

The supplied dataset contains

- 500 training trajectories,
- 20 validation trajectories, and
- 20 held-out test trajectories.

Each trajectory contains 600 states separated by $\Delta t_{\text{data}} = 0.01$ s, corresponding to physical times 0, 0.01, 0.02, $\dots$, 5.99 s. For a trajectory with N mesh nodes and M triangular cells, the following quantities are provided:

Quantity	Shape	Description
<code>mesh_pos</code>	$(N, 2)$	Node coordinates (x, y)
<code>cells</code>	$(M, 3)$	Node indices of triangular cells
<code>node_type</code>	(N)	Node type
<code>velocity</code>	$(600, N, 2)$	Velocity (u_x, u_y) at every state

The number of nodes and the mesh connectivity may differ between trajectories. Within a single trajectory, however, the mesh is fixed in time.

The node types used in this assignment are

- 0 : interior / normal node,
- 4 : inflow node,
- 5 : outflow node,
- 6 : wall boundary node.

Interior and outflow nodes are treated as dynamic prediction variables.

For each trajectory, the inflow and wall velocities are prescribed boundary values and are constant in time. During an autoregressive rollout, these prescribed boundary values should therefore be restored after every prediction step.

1.3 Prediction task

The required prediction horizon is $\Delta t_{\text{pred}} = 1$ s. Since the stored states are separated by 0.01 s, the input and target states are separated by 100 stored time steps.

If x_t denotes the flow state at physical time t , examples of valid training pairs are

$$(x_{0.00}, x_{1.00}), \quad (x_{0.01}, x_{1.01}), \quad (x_{0.02}, x_{1.02}), \quad \dots$$

and, later in the same trajectory,

$$(x_{1.00}, x_{2.00}), \quad (x_{1.05}, x_{2.05}), \quad \dots, \quad (x_{4.99}, x_{5.99}).$$

Therefore, **1 second is the separation between an input and its target; it is not the spacing between consecutive training examples.**

A trajectory containing 600 states provides $600 - 100 = 500$ possible one-second input–target pairs.

The implementation of data loading and dataset class are given in the assignment’s notebook.

1.4 Preprocessing and normalization

You may choose an appropriate normalization or standardization procedure for the variables used by your model.

Any statistics used for normalization must be estimated **using the training split only**. The same statistics must then be applied unchanged to validation and test data.

You should not use the test set for preprocessing decisions, hyperparameter tuning, or model selection.

1.5 Part I: Deterministic model

Train a deterministic model that predicts the velocity field one second into the future.

One possible formulation is to predict the velocity increment

$$\Delta \mathbf{u}_t = \mathbf{u}_{t+1s} - \mathbf{u}_t,$$

although predicting the future velocity field directly is also possible.

A standard deterministic baseline minimizes mean-squared error over the dynamic nodes:

$$\mathcal{L}_{\text{det}} = \frac{1}{N_{\text{dyn}}} \sum_{i \in \mathcal{D}} \|\hat{\mathbf{u}}_i(t+1) - \mathbf{u}_i(t+1)\|_2^2,$$

where $\mathcal{D}$ denotes the set of interior and outflow nodes.

1.6 Part II: Conditional flow matching

Train a conditional flow-matching model for the one-second future velocity field, or equivalently for the one-second velocity increment.

The objective is to model a conditional distribution such as

$$p(\Delta \mathbf{u}_t \mid \mathcal{C}_t),$$

where $\mathcal{C}_t$ contains the information available at the current state, for example the current velocity field, mesh geometry, connectivity, and node types.

For a standard flow-matching construction, let $\mathbf{x}_0 \sim \mathcal{N}(0, I)$ denote a noise sample and let $\mathbf{x}_1$ denote the target quantity to be modeled. Draw $\tau \sim U[0, 1]$ and construct $\mathbf{x}_\tau = (1-\tau)\mathbf{x}_0 + \tau\mathbf{x}_1$. The corresponding target velocity field in flow-matching space is $\mathbf{v}^* = \mathbf{x}_1 - \mathbf{x}_0$. A conditional model $\mathbf{v}_\theta(\mathbf{x}_\tau, \tau \mid \mathcal{C}_t)$ is trained to approximate this vector field.

At inference time, samples are generated by starting from fresh noise and integrating the learned flow from $\tau = 0$ to $\tau = 1$.

You may choose the numerical integration method and the number of integration steps.

As for the deterministic model, prediction losses should only be applied to dynamic nodes.

1.7 Autoregressive evaluation

Both models must be evaluated through autoregressive rollouts. The notebook provides the rollout loops, metric computation, aggregation, and plotting code. Students must implement the deterministic prediction adapter and the conditional flow-matching sampling adapter, then run the supplied evaluation.

Start from the true state at $t = 0$ and generate model states at the nominal times 0, 1, 2, 3, 4, 5, 6 s. For the deterministic model, this produces one rollout for each initial condition.

For the flow-matching model, generate 5 independent sampled rollouts for each test trajectory.

1.7.1 Evaluation metric: wake temporal activity

The models will be evaluated using the temporal activity of the velocity field in the downstream wake.

For two states at times t_0 and t_1 , first define at each mesh node i

$$a_i(t_0, t_1) = \frac{\|\mathbf{u}_i(t_1) - \mathbf{u}_i(t_0)\|_2}{t_1 - t_0}.$$

The full two-dimensional velocity vector is used, so changes in both velocity magnitude and velocity direction contribute to the metric.

This quantity is first computed at the mesh nodes. Triangles are then used to perform an area-weighted spatial average, which avoids biasing the metric toward regions where the nonuniform mesh happens to contain more nodes.

For a triangular cell K with vertices i , j , and k , define

$$a_K = \frac{a_i + a_j + a_k}{3}.$$

Let A_K denote the area of triangle K . The wake temporal activity is

$$D_{\text{wake}}(t_0, t_1) = \frac{\sum_{K \in \Omega_{\text{wake}}} A_K a_K}{\sum_{K \in \Omega_{\text{wake}}} A_K}$$

where Ω_{wake} contains triangles whose centroids lie inside the fixed region

$$0.55 \leq x \leq 1.55, \quad 0.03 \leq y \leq 0.39.$$

A larger value of D_{wake} indicates stronger finite-time change of the velocity field in the wake.

Burn-in interval

The first rollout interval, $0 \rightarrow 1$ s, is treated as a burn-in interval and is not included in the final metric. The evaluated intervals are therefore $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $4 \rightarrow 5$, $5 \rightarrow 6$ s. The stored ground-truth trajectory ends at 5.99 s rather than exactly 6.00 s. Consequently, for the final ground-truth interval, use $5.00 \rightarrow 5.99$ s and divide by the actual interval length, 0.99 s. The model rollouts use their nominal $5.00 \rightarrow 6.00$ s states and therefore use an interval length of 1.00 s.

1.7.2 Aggregation over test trajectories

The supplied evaluation code evaluates the models on the 20 held-out test trajectories. For ground truth and the deterministic model, it computes one temporal-activity value for each trajectory and evaluated time interval.

For flow matching, the evaluator calls the student-implemented sampling adapter to produce five sampled rollouts per test trajectory, then computes the metric separately for each rollout.

The supplied aggregation function first averages the five flow-matching temporal-activity values within each trajectory and time interval. It then computes the mean and standard deviation across test trajectories. The sampled velocity fields themselves are not averaged before computing the metric.

The supplied plotting function displays the mean temporal-activity curves for ground truth, the deterministic model, and the flow-matching model, with optional shading showing one standard deviation across trajectories.

Students should run these functions after implementing the model adapters and training their models, then interpret the resulting comparison.

1.8 Implementation freedom

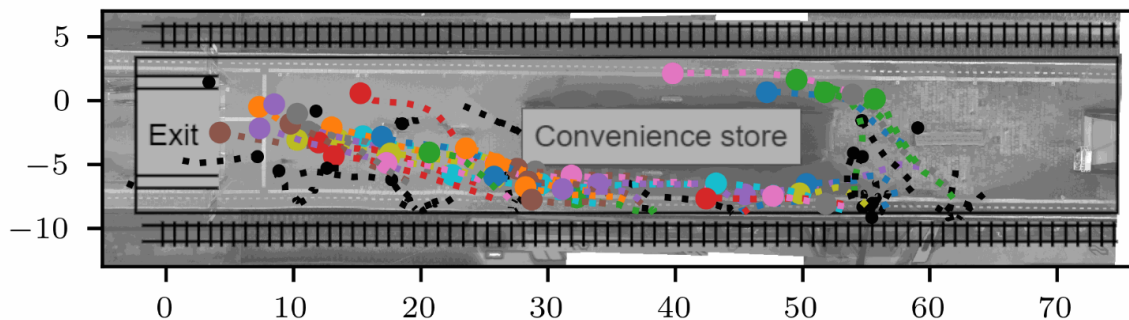
The following choices are left to you:

- neural-network architecture;
- representation of the triangular mesh;
- batching strategy for variable-size meshes;
- input and target normalization;
- optimizer and learning-rate schedule;
- representation of the conditioning information;
- numerical solver and number of integration steps used for flow-matching sampling.

Your choices should be justified briefly and should be applied consistently when comparing the deterministic and flow-matching models.

2 Lagrangian Systems: Pedestrians on Eindhoven Centraal

[50pt]



2.1 Task Description

For the second task, you will model the future motion of pedestrians walking on platforms 3 and 4 of Eindhoven Centraal train station. You can access the notebook for this task [here](#).

Given the first 10 observed frames of a scene, your task is to generate the remaining 20 frames for all pedestrians in that scene. This is a conditional simulation problem. The future motion of a crowd is uncertain, so the model should represent a distribution over plausible futures rather than a single prediction.

You will construct and compare two conditional flow matching models that both use time bundling:

1. a model with an uninformed Gaussian prior on the future velocities and
2. a model with an informed, data dependent prior.

The main objective is to investigate how the choice of prior affects the generated trajectories.

The focus of this assignment is not on model performance. The focus is on an architecture that is well aligned with the problem formulation and on a careful comparison between the uninformed and the informed prior. Every group member must understand the model, be able to explain why it is designed the way it is, and be able to interpret the evaluation results.

The dataset can be downloaded from this [link](#).

2.2 Dataset

The dataset consists of 100 000 samples. Each sample is a scene containing the 2D positions of 5 to 100 pedestrians over 30 frames, covering 15 s with a fixed frame spacing of $\Delta t = 0.5$ s. In total, the dataset contains approximately 2.4 million pedestrian trajectories. Positions are given in metres.

A sample with N pedestrians is stored as an array of shape $(N, 30, 2)$. Every pedestrian in a sample is present in all 30 frames. The number of pedestrians differs between samples.



2.3 Prediction task

Let $\mathbf{p}_i^t \in \mathbb{R}^2$ denote the position of pedestrian i at frame $t \in \{0, \dots, 29\}$, and define its velocity as

$$\mathbf{v}_i^t = \frac{\mathbf{p}_i^t - \mathbf{p}_i^{t-1}}{\Delta t}.$$

The model observes the first $H = 10$ frames and generates the remaining $K = 20$ frames:

$$\text{condition: } c = \{\mathbf{p}_i^t\}_{t=0, \dots, 9}^{i=1, \dots, N}, \quad \text{target: } x_1 = (\mathbf{v}^{10}, \dots, \mathbf{v}^{29}) \in \mathbb{R}^{N \times 20 \times 2}.$$

The condition may also include quantities derived from the observed frames, such as velocities and accelerations.

All 20 target frames are generated in a single model call (time bundling). Positions are recovered from the generated velocities, starting at the last observed position:

$$\mathbf{p}_i^{9+k} = \mathbf{p}_i^9 + \Delta t \sum_{j=1}^k \mathbf{v}_i^{9+j}, \quad k = 1, \dots, 20.$$

2.4 Data splits and preprocessing

Split the dataset randomly at sample level into a training, validation and test set. The held out test set must contain **at least 1 000 samples** drawn randomly from the dataset, a larger test set is preferred. Report the random seed and the size of each split.

You may choose an appropriate normalization or standardization procedure. Any statistics used for normalization must be estimated using the training split only and applied unchanged to validation and test data. The test set must not be used for preprocessing decisions, hyperparameter tuning, model selection or of course model training.

2.5 Model architecture

The model must learn patterns along two axes: **space**, meaning the interactions between pedestrians within a frame, and **time**, meaning how the motion of each pedestrian evolves over the frames. As discussed in Practical 4, message passing captures the spatial axis but has no mechanism for the temporal axis, while flattening all frames into a single vector ignores temporal order and locality and bases the interactions on a single frame.

Focus on designing an architecture that is well suited to both axes. Justify your design choices with respect to the problem formulation.

Use the same architecture, training budget and sampling procedure for both models, so that the prior is the only difference between them.

2.6 Part I: Uninformed Gaussian prior

Train a conditional flow matching model for the target x_1 using the **independent coupling**

$$\pi(x_0, x_1 \mid c) = p_0(x_0) p_1(x_1 \mid c), \quad p_0 = \mathcal{N}(0, \sigma^2 I).$$

The prior samples do not depend on the condition. The noise spread σ is a hyperparameter.

2.7 Part II: Informed prior

Train a second conditional flow matching model using a **data dependent coupling**

$$\pi(x_0, x_1 | c) = p_0(x_0 | c) p_1(x_1 | c).$$

This factorization requires the prior to be conditionally independent of the target given the observed frames, $x_0 \perp x_1 | c$. The prior may therefore only be constructed from the observed frames $t = 0, \dots, 9$. The target frames are unavailable at inference, so using them to construct the prior leaks information and is not allowed.

The prior must remain stochastic and must be sampled in the same way during training and inference. You are free to design your own informed prior, but you have to motivate why it should increase performance, at least in theory. It is not a problem if the informed prior performs worse than the uninformed prior, as long the implementation, evaluation and reflection are performed well.

2.8 Evaluation

Evaluate both models on the held out test set. For every test sample, generate $M = 20$ independent sampled futures. All metrics are computed on the generated frames $t \in \mathcal{T} = \{10, \dots, 29\}$ only. Extending this evaluation with other, even more informative measures will be a way to earn bonus points.

2.8.1 Pointwise metrics

For test sample s with N_s pedestrians and generated future m , let $d_{s,m,i}^t = \|\hat{\mathbf{p}}_{s,m,i}^t - \mathbf{p}_{s,i}^t\|_2$ denote the displacement error of pedestrian i at frame t . Define

$$\text{ADE}_{s,m} = \frac{1}{N_s K} \sum_{i=1}^{N_s} \sum_{t \in \mathcal{T}} d_{s,m,i}^t, \quad \text{FDE}_{s,m} = \frac{1}{N_s} \sum_{i=1}^{N_s} d_{s,m,i}^{29}.$$

With S test samples, report

$$\begin{aligned} \text{ADE} &= \frac{1}{SM} \sum_{s=1}^S \sum_{m=1}^M \text{ADE}_{s,m}, & \text{FDE} &= \frac{1}{SM} \sum_{s=1}^S \sum_{m=1}^M \text{FDE}_{s,m}, \\ \min_{20} \text{ADE} &= \frac{1}{S} \sum_{s=1}^S \min_m \text{ADE}_{s,m}, & \min_{20} \text{FDE} &= \frac{1}{S} \sum_{s=1}^S \min_m \text{FDE}_{s,m}. \end{aligned}$$

The minimum is taken at data sample level: for each test sample, one generated future is selected based on its error averaged over all pedestrians in the scene. It is not the best future per pedestrian.

2.8.2 Distributional metrics

Provide histogram density plots comparing the ground truth with both models for the following quantities, pooled over all pedestrians, frames and generated futures:

- **Velocity magnitude** $\|\mathbf{v}_i^t\|_2$.
- **Acceleration magnitude** $\|\mathbf{a}_i^t\|_2$, with $\mathbf{a}_i^t = (\mathbf{v}_i^t - \mathbf{v}_i^{t-1})/\Delta t$.

- **Tortuosity**, the ratio of path length to net displacement over the prediction window,

$$\chi_i = \frac{\sum_{t \in \mathcal{T}} \|\mathbf{p}_i^t - \mathbf{p}_i^{t-1}\|_2}{\max(\|\mathbf{p}_i^{29} - \mathbf{p}_i^9\|_2, 0.01)}.$$

A straight path gives $\chi_i = 1$. This quantity measures how curved paths are.

- **Distance to closest neighbour** within the same frame, $r_i^t = \min_{j \neq i} \|\mathbf{p}_i^t - \mathbf{p}_j^t\|_2$, where j runs over the pedestrians of the same sample and the same generated future. The x-axis of the histogram plot has to range from $r = 0$ to $r = 2$ meters.

2.8.3 Comparison

Report the four pointwise metrics for both models, together with the four histogram density plots. Compare the uninformed and the informed prior, and interpret the differences in both the pointwise and the distributional metrics in relation to your design choices.

2.9 Implementation freedom

The following choices are left to you:

- neural network architecture, subject to the requirements above
- graph construction between pedestrians
- representation of the conditioning information
- input and target normalization
- noise scale σ and design of the informed prior
- optimizer and learning rate schedule
- numerical solver and number of integration steps used for sampling
- sizes of the training and validation splits, subject to the test set constraint.

Your choices should be justified briefly and should be applied consistently when comparing the two models.

3 Deliverables

The assignment has 2 deliverables:

1. Two Jupyter notebooks (part 1 and part 2) to be uploaded in Ans that both contain the required plots and results in the output cells. Explanations of the implementation or the results are not required within the notebook itself, they should be written in Ans. Code comments are not required, but appreciated. You do not have to train the models within the notebooks per se, as long as the provided notebooks contain the results that are given in Ans. If we have doubts about the validity of the results, we will ask the group to provide model weights for reproducibility.
2. In Ans, all assignment questions should be answered. These questions will be based on the task descriptions written above. We will ask you to upload the results, tables and/or plots that back up your explanations from the notebook, on Ans. The information put

in Ans will be seen as ground truth, so make sure you fill in the latest and correct information, aligned with your notebook.